



# CIBERSEGURIDAD EN ENTORNOS DE LAS TECNOLOGÍAS DE LA INFORMACIÓN

Puesta en Producción Segura

## TEMA 4

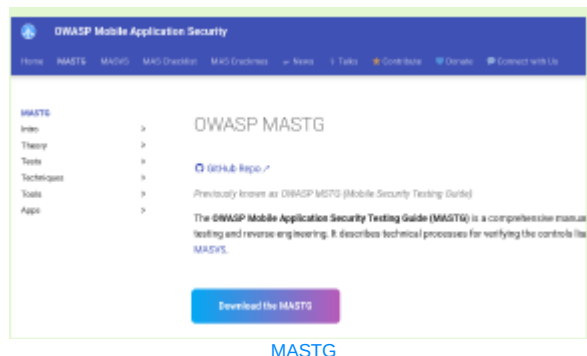
Detección de problemas de seguridad en aplicaciones para dispositivos móviles  
Alberto Diéguez Álvarez

## Contenido

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Descripción de la tarea.....</b>                   | <b>3</b>  |
| <b>Apartado 1: MASTG (ver enlace).....</b>            | <b>4</b>  |
| <b>Apartado 2: CASB y MDM.....</b>                    | <b>13</b> |
| <b>Apartado 3: Diseño y capacidades de CASB .....</b> | <b>14</b> |

## Descripción de la tarea.

### Caso práctico



**Julián** ha estado trabajando en un proyecto de una aplicación móvil junto con una solución CASB que quiere salir al mercado y ser revolucionaria. Para comprobar que la aplicación es segura se ha seguido la última versión de la Guía de Pruebas de Seguridad de Aplicaciones Móviles (MASTG) desarrollada por OWASP.

**Julián** entiende que el Estándar de Verificación de Seguridad de Aplicaciones Móviles (MASVS) proporciona un marco claro y detallado que aborda los requisitos de seguridad esenciales para el desarrollo y la evaluación de aplicaciones móviles. Por otro lado, la Guía de Pruebas de Seguridad de Aplicaciones Móviles (MASTG) se alinea estrechamente con los requisitos establecidos por la MASVS, ofreciendo un conjunto adicional de directrices específicas para realizar pruebas de seguridad efectivas en aplicaciones móviles. La combinación de ambas herramientas, MASVS y MASTG, proporciona un enfoque integral para evaluar y mejorar la seguridad en aplicaciones móviles, permitiendo a los profesionales adaptar sus estrategias según el contexto específico.

Uno de los puntos que más se han trabajado durante el proyecto de la solución CASB es ser diferenciadores y poder corregir los problemas de adopción que han tenido las soluciones de MDM (Mobile Device Management). Saben que las soluciones para móviles tienen que ser capaces de lidiar con varios problemas como la privacidad, (*Bring Your Own Device*), etc. BYOD

# ¿Qué te pedimos que hagas?

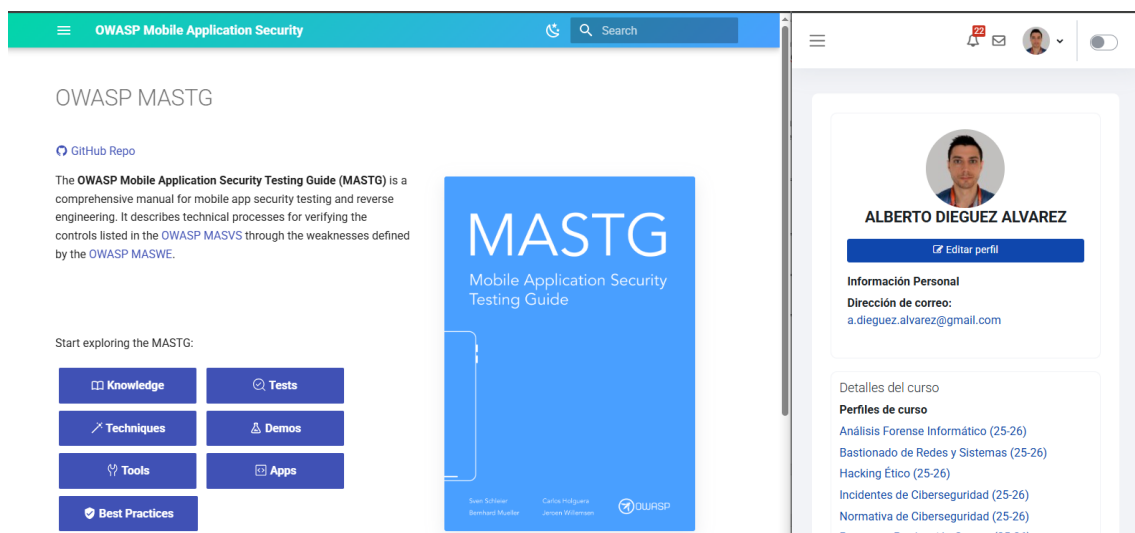
Esta tarea es eminentemente teórica donde el alumno deberá responder y desarrollar una serie de preguntas. El alumno debe saber manejar guías para comprobar si una aplicación móvil es segura o no, entender distintos conceptos como CASB, MDM, BYOD, etc. y ser capaz de entender que capacidades aportan las soluciones tecnológicas, con qué problemas se encuentran en el mercado y qué capacidades adicionales podrían tener y qué las empresas valorarían.

## Apartado 1: MASTG (ver [enlace](#))

En la unidad 2 ya vimos que OWASP ha desarrollado el proyecto Mobile Application Security (MAS) que proporciona un estándar de seguridad para aplicaciones móviles (OWASP MASVS) y una guía de pruebas exhaustiva (OWASP MASTG) que cubre los procesos, técnicas y herramientas utilizados durante una prueba de seguridad de aplicaciones móviles, así como un conjunto exhaustivo de casos de prueba que permite a los probadores ofrecer resultados coherentes y completos. En este apartado se pide **revisar la guía MASTG**, centrándote en las **diferentes defensas contra la Ingeniería Inversa en Android** (Android Anti-Reversing Defenses) y **en IOS** (IOS Anti-Reversing Defenses)

### 1. Una medida defensiva es comprobar el "Rooteado" en Android y el "Jailbreak" en IOS. Rellena la siguiente tabla utilizando la guía MASTG

Entro en la web MASTG y busco información para contestar a las preguntas. Adjunto fotos.



OWASP Mobile Application Security

Search

MASVS-RESILIENCE

ANDROID

KNOW

MASTG-KNOW-0027: Root Detection

In the context of anti-reversing, the goal of root detection is to make running the app on a rooted device a bit more difficult, which in turn blocks some of the tools and techniques reverse engineers like to use. Like most other defenses, root detection is not very effective by itself, but implementing multiple root checks that are scattered throughout the app can improve the effectiveness of the overall anti-tampering scheme.

For Android, we define "root detection" a bit more broadly, including custom ROMs detection, i.e., determining whether the device is a stock Android build or a custom build.

Root detection can also be implemented through libraries such as [RootBeer](#) or [Android RASP](#) (none of them endorsed by OWASP, see the disclaimers in their respective sections). These libraries implement multiple root detection techniques using both Java and native code to make bypassing more difficult. They also provide sample apps to demonstrate their capabilities, see [RootBeer Sample](#) and [Android RASP Sample](#).

### File Existence Checks

Perhaps the most widely used method of programmatic detection is checking for files typically found on rooted devices, such as package files of common rooting apps and their associated files and directories, including the following:

```
/system/app/Superuser.apk
/system/etc/init.d/99SuperSUDaemon
/dev/com.koushikdutta.superuser.daemon/
```

OWASP Mobile Application Security

Search

MASVS-RESILIENCE

IOS

KNOW

MASTG-KNOW-0084: Jailbreak Detection

Jailbreak detection mechanisms are added to reverse engineering defense to make running the app on a jailbroken device more difficult. This blocks some of the tools and techniques reverse engineers like to use. Like most other types of defense, jailbreak detection is not very effective by itself, but scattering checks throughout the app's source code can improve the effectiveness of the overall anti-tampering scheme.

You can learn more about Jailbreak/Root Detection in the research study ["Jailbreak/Root Detection Evasion Study on iOS and Android"](#) by Dana Geist and Marat Nigmatullin.

### Common Jailbreak Detection Checks

Here we present three typical jailbreak detection techniques:

**File-based Checks:**

The app might be checking for files and directories typically associated with jailbreaks, such as:

```
/Applications/Cydia.app
/Applications/FakeCarrier.app
/Applications/Icy.app
/Applications/IntelliScreen.app
/Applications/MxTube.app
/Applications/RockApp.app
/Applications/SSSettings.app
```

OWASP Mobile Application Security

Search

MASVS-RESILIENCE

ANDROID

Android Anti-Reversing Defenses

Table of contents

Overview

Knowledge Articles

Overview

**General Disclaimer:**

The lack of any of these measures does not cause a vulnerability - instead, they are meant to increase the app's resilience against reverse engineering and specific client-side attacks.

None of these measures can assure a 100% effectiveness, as the reverse engineer will always have full access to the device and will therefore always win (given enough time and resources)!

For example, preventing debugging is virtually impossible. If the app is publicly available, it can be run on an untrusted device that is under full control of the attacker. A very determined attacker will eventually manage to bypass all the app's anti-debugging controls by patching the app binary or by dynamically modifying the app's behavior at runtime with tools such as Frida.

You can learn more about principles and technical risks of reverse engineering and code modification in these OWASP documents:

- [OWASP Architectural Principles That Prevent Code Modification or Reverse Engineering](#)

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

Página 5 | 15

[illegible]

**2. Otra medida defensiva es la ofuscación. Rellena la siguiente tabla utilizando la guía MASTG**

OWASP Mobile Application Security

MASTG-KNOW-0111: Obfuscation

Obfuscation is the process of transforming code and data to make it more difficult to comprehend (and sometimes even difficult to disassemble). It is usually an integral part of the software protection scheme. Obfuscation isn't something that can be simply turned on or off, programs can be made incomprehensible, in whole or in part, in many ways and to different degrees.

Note

All the presented techniques below will not stop someone with enough time and budget from reverse engineering your app. However, combining these techniques will make their job significantly harder. The aim is thus to discourage reverse engineers from performing further analysis and not making it worth the effort.

The following techniques can be used to obfuscate an application:

- Name obfuscation
- Instruction substitution
- Control flow flattening
- Dead code injection
- String encryption
- Packing

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

OWASP Mobile Application Security

MASTG-KNOW-0089: Obfuscation

Obfuscation introduces several well-known obfuscation techniques that can be used in mobile apps in general.

Name Obfuscation

The standard compiler generates binary symbols based on class and function names from the source code. Therefore, if no obfuscation was applied, symbol names remain meaningful and can be easily read straight from the app binary. For instance, a function which detects a jailbreak can be located by searching for relevant keywords (e.g. "jailbreak"). The listing below shows the disassembled function `JailbreakDetectionViewController.jailbreakTest4Tapped` from the `DVIA-v2`.

```
__T07DVIA_v232JailbreakDetectionViewControllerC20jailbreakTest4TappedyyF:  
stp    x22, x21, [sp, #-0x30]!  
mov    rbp, rsp
```

After the obfuscation we can observe that the symbol's name is no longer meaningful as shown on the listing below.

```
__T07DVIA_v232zNhtWKOptikYUBNBgfFVMjSkvRdhnbbyFySbyypF:  
stp    x22, x21, [sp, #-0x30]!  
mov    rbp, rsp
```

Nevertheless, this only applies to the names of functions, classes and fields. The actual code remains unmodified, so an attacker can still read the disassembled version of the function and try to understand its purpose (e.g. to retrieve the logic of a security

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

OWASP Mobile Application Security

MASTG-TOOL-0022: ProGuard

ProGuard is a free Java class file shrinker, optimizer, obfuscator, and preverifier. It detects and removes unused classes, fields, methods, and attributes and can also be used to delete logging-related code.

Techniques

MASTG-TECH-0129: Verifying Android Dependencies at Runtime

Tests

MASTG-TEST-0003: Testing Logs for Sensitive Data

MASTG-TEST-0042: Checking for Weaknesses in Third Party Libraries

© OWASP Foundation 2026. This work is licensed under CC-BY 4.0. For any reuse or distribution, you must make clear to others the license terms of this work. OWASP is a registered trademark of the OWASP Foundation, Inc. This website uses cookies to analyze our traffic and only share that information with our analytics partners. Learn more.

Made with Material for MkDocs | Website and covers designed by Carlos Holguera.

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:  
a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

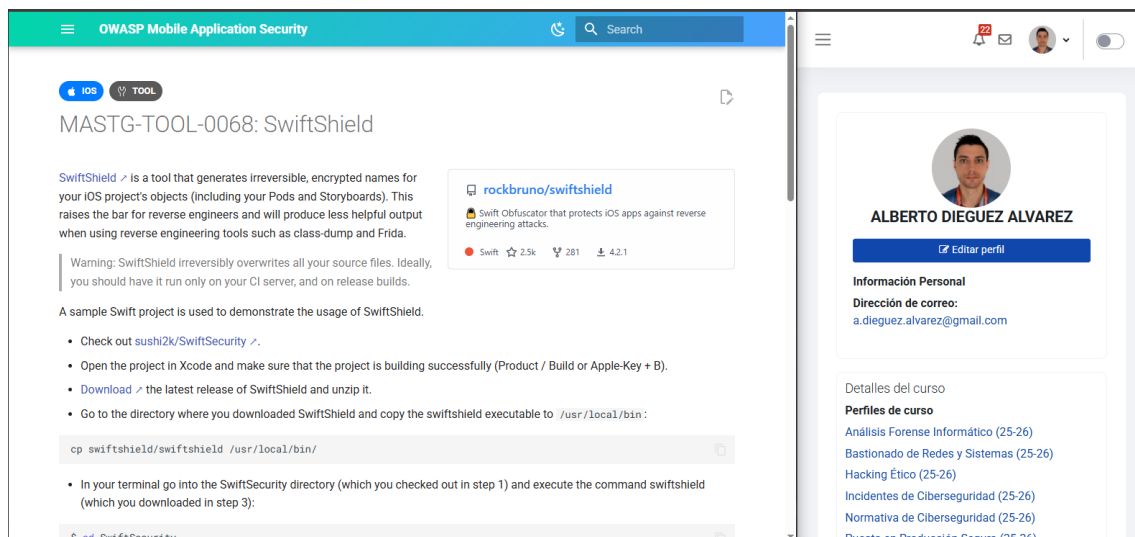
Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)





|                                                                         |                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿En qué consiste la ofuscación?                                         | Es el proceso de transformar código y datos para dificultar su comprensión. Se suele utilizar para la protección del software, no es algo que se pueda activar o desactivar fácilmente, los programas pueden volverse incomprensibles.            |
| Indica para que se utiliza la herramienta ProGuard y cómo se utiliza.   | ProGuard es una herramienta de ofuscación, optimización y reducción de código en JAVA y Android. Se utiliza para detectar y eliminar clases, campos, métodos y atributos no utilizados, también para eliminar código relacionado con el registro. |
| Indica para que se utiliza la herramienta SwiftShield y cómo se utiliza | SwiftShield es una herramienta que genera nombres cifrados e irreversibles para los objetos de tu proyecto iOS. En resumen, es una herramienta de ofuscación para aplicaciones iOS escritas en Swift.                                             |



3. Explorar aplicaciones utilizando un depurador es una técnica muy poderosa. No solo se puede rastrear variables que contienen datos confidenciales y modificar el flujo de control de la aplicación, sino también leer y modificar la memoria y los registros. Rellena la siguiente tabla utilizando la guía MASTG

MASTG-KNOW-0028: Anti-Debugging

Messing with JDWP-Related Data Structures

In Dalvik, the global virtual machine state is accessible via the `DvmGlobals` structure. The global variable `gDvm` holds a pointer to this structure. `DvmGlobals` contains various variables and pointers that are important for JDWP debugging and can be tampered with.

```
struct DvmGlobals {
    /*
     * Some options that could be worth tampering with :)
     */
    bool    jdwpAllowed;    // debugging allowed for this process?
    bool    jdwpConfigured; // has debugging info been provided?
    JdwpTransportType jdwpTransport;
    bool    jdwpServer;
    char*   jdwpHost;
    int     jdwpPort;
    bool    jdwpSuspend;

    Thread* threadList;

    bool    nativeDebuggerActive;
    bool    debuggerConnected; // debugger or DDMS is connected */
    bool    debuggerActive;    // debugger is making requests */
    JdwpState* jdwpState;
};
```

For example, setting the `gDvm.methDalvikDdmcServer_dispatch` function pointer to NULL crashes the JDWP thread `^`:

```
JNIEXPOR (boolean JNICALL Java_poc_c_crashOnInit ( JNIEnv* env , jobject ) {
    gDvm.methDalvikDdmcServer_dispatch = NULL;
}
```

MASTG-KNOW-0085: Anti-Debugging Detection

Using ptrace

As seen in [Debugging](#), the iOS XNU kernel implements a `ptrace` system call that's lacking most of the functionality required to properly debug a process (e.g. it allows attaching/stepping but not read/write of memory and registers).

Nevertheless, the iOS implementation of the `ptrace` syscall contains a nonstandard and very useful feature: preventing the debugging of processes. This feature is implemented as the `PT_DENY_ATTACH` request, as described in the [official BSD System Calls Manual](#) `^`. In simple words, it ensures that no other debugger can attach to the calling process; if a debugger attempts to attach, the process will terminate. Using `PT_DENY_ATTACH` is a fairly well-known anti-debugging technique, so you may encounter it often during iOS pentests.

Before diving into the details, it is important to know that `ptrace` is not part of the public iOS API. Non-public APIs are prohibited, and the App Store may reject apps that include them. Because of this, `ptrace` is not directly called in the code; it's called when a `ptrace` function pointer is obtained via `disym`.

The following is an example implementation of the above logic:

```
#import <dlfcn.h>
#import <sys/types.h>
#import <stdio.h>
typedef int (*ptrace_ptr_t)(int _request, pid_t _pid, caddr_t _addr, int _data);
void anti_debug() {
    ptrace_ptr_t ptrace_ptr = (ptrace_ptr_t)disym(RTLD_SELF, "ptrace");
    ptrace_ptr(31, 0, 0, 0); // PT_DENY_ATTACH = 31
}
```

**Bypass:** To demonstrate how to bypass this technique we'll use an example of a disassembled binary that implements this approach:

```
__text:00019074      MOVW     R1, #lower16:(@Ptrace - 0x19088) ; 'ptrace'
```

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:

a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

Página 9 | 15

MASTG-KNOW-0028: Anti-Debugging

Search

### JDWP Anti-Debugging

In the chapter "Reverse Engineering and Tampering", we talked about JDWP, the protocol used for communication between the debugger and the Java Virtual Machine. We showed that it is easy to enable debugging for any app by patching its manifest file, and changing the `ro.debuggable` system property which enables debugging for all apps. Let's look at a few things developers do to detect and disable JDWP debuggers.

#### Checking the Debuggable Flag in ApplicationInfo

We have already encountered the `android:debuggable` attribute. This flag in the Android Manifest determines whether the JDWP thread is started for the app. Its value can be determined programmatically, via the app's `ApplicationInfo` object. If the flag is set, the manifest has been tampered with and allows debugging.

```

public static boolean isDebuggable(Context context){

    return ((context.getApplicationContext().getApplicationInfo().flags & ApplicationInfo.FLAG_DEBUGGABLE) != 0);
}

```

#### isDebuggerConnected

While this might be pretty obvious to circumvent for a reverse engineer, you can use `isDebuggerConnected` from the `android.os.Debug` class to determine whether a debugger is connected.

```

public static boolean detectDebugger() {
    return Debug.isDebuggerConnected();
}

```

**ALBERTO DIEGUEZ ALVAREZ**

Editar perfil

**Información Personal**

**Dirección de correo:**  
a.dieguez.alvarez@gmail.com

**Detalles del curso**

**Perfiles de curso**

- Análisis Forense Informático (25-26)
- Bastionado de Redes y Sistemas (25-26)
- Hacking Ético (25-26)
- Incidentes de Ciberseguridad (25-26)
- Normativa de Ciberseguridad (25-26)
- Puesta en Producción Segura (25-26)

|                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Explica qué consiste la técnica anti depuración JDWP en Android                 | Es un protocolo utilizado para la comunicación entre el depurador y la máquina virtual Java. Consiste en detectar o bloquear la conexión de un depurador JDWP para impedir el análisis dinámico de la app.                                                                                                                                                                                   |
| Explica qué es ptrace y cómo se puede utilizar para evitar la depuración en IOS | El kernel de XNU de iOS implementa una llamada al sistema ptrace, esta contiene una característica no estándar y muy útil: impide la depuración de procesos. Ptrace permite a un proceso controlar y observar a otro proceso. Para evitar la depuración en iOS se utiliza con la opciones <b>ptrace(PT_DENY_ATTACH, 0, 0, 0,);</b> esto impide que un depurador pueda adjuntarse al proceso. |

4. La presencia de herramientas, frameworks y aplicaciones comúnmente utilizadas por la ingeniería inversa puede indicar un intento de realizar ingeniería inversa en la aplicación. Algunas de estas herramientas sólo pueden ejecutarse en un dispositivo con jailbreak o rooteado, mientras que otras obligan a la aplicación a entrar en modo de depuración o dependen del inicio de un servicio en segundo plano en el teléfono móvil. Por lo tanto, existen diferentes formas que una aplicación puede implementar para detectar un ataque de ingeniería inversa y reaccionar ante él, por ejemplo, finalizándose ella misma. Utilizando la guía MASTG

The image displays two screenshots of the OWASP Mobile Application Security (OWASP MAS) website, specifically the MASTG (Mobile Application Security Testing Guide) section.

**Top Screenshot: MASTG-TOOL-0031: Frida**

- The page title is "MASTG-TOOL-0031: Frida".
- The description states: "Frida is a free and open source dynamic code instrumentation toolkit written by Ole André Vadla Ravnås that works by injecting the QuickJS JavaScript engine (previously Duktape and V8) into the instrumented process. Frida lets you execute snippets of JavaScript into native apps on Android and iOS (as well as on other platforms)."
- A large red "FRIDA" logo is prominently displayed.
- The "Installation" section instructs: "To install Frida locally, simply run:" followed by the command: `pip install frida-tools`.

**Bottom Screenshot: MASTG-TOOL-0001: Frida (Android)**

- The page title is "MASTG-TOOL-0001: Frida (Android)".
- The description states: "Frida supports interaction with the Android Java runtime through the Java API. You'll be able to hook and call both Java and native functions inside the process and its native libraries. Your JavaScript snippets have full access to memory, e.g. to read and/or write any structured data."
- A list of tasks that Frida APIs offer is provided:
  - Instantiate Java objects and call static and non-static class methods (Java API).
  - Replace Java method implementations (Java API).
  - Enumerate live instances of specific classes by scanning the Java heap (Java API).
  - Scan process memory for occurrences of a string (Memory API).
  - Intercept native function calls to run your own code at function entry and exit (Interceptor API).
- A note mentions: "Remember that on Android, you can also benefit from the built-in tools provided when installing Frida, which include the Frida CLI (frida), frida-ps, frida-ls-devices, and frida-trace, to name some of them."
- A comparison is made: "Frida is often compared to Xposed or LSPosed. However, this comparison is far from fair, as both frameworks were designed with different goals in mind. This is important to understand as an app security tester so that you can know which framework to use in which situation:"

OWASP Mobile Application Security

Search

IOS

TOOL

MASTG-TOOL-0039: Frida (iOS)

Frida supports interaction with the Objective-C runtime through the [ObjC API](#). You'll be able to hook and call both Objective-C and native functions inside the process and its native libraries. Your JavaScript snippets have full access to memory, e.g. to read and/or write any structured data.

frida/frida

Clone this repo to build Frida

Meson

19.8k

2k

Frida 17.7.3

Here are some tasks that Frida APIs offers and are relevant or exclusive on iOS:

- Instantiate Objective-C objects and call static and non-static class methods ([ObjC API](#)).
- Trace Objective-C method calls and/or replace their implementations ([Interceptor API](#)).
- Enumerate live instances of specific classes by scanning the heap ([ObjC API](#)).
- Scan process memory for occurrences of a string ([Memory API](#)).
- Intercept native function calls to run your own code at function entry and exit ([Interceptor API](#)).

Remember that on iOS, you can also benefit from the built-in tools provided when installing Frida, which include the Frida CLI ([frida](#)), [frida-ps](#), [frida-ls-devices](#) and [frida-trace](#), to name a few.

There's a [frida-trace](#) feature exclusive on iOS worth highlighting: tracing Objective-C APIs using the `-m` flag and wildcards. For example, tracing all methods including "HTTP" in their name and belonging to any class whose name starts with "NSURL" is as easy as running:

OWASP Mobile Application Security

Search

MASVS-RESILIENCE

ANDROID

IT KNOW

MASTG-KNOW-0030: Detection of Reverse Engineering Tools

The presence of tools, frameworks and apps commonly used by reverse engineers may indicate an attempt to reverse engineer the app. Some of these tools can only run on a rooted device, while others force the app into debugging mode or depend on starting a background service on the mobile phone. Therefore, there are different ways that an app may implement to detect a reverse engineering attack and react to it, e.g. by terminating itself.

You can detect popular reverse engineering tools that have been installed in an unmodified form by looking for associated application packages, files, processes, or other tool-specific modifications and artifacts. In the following examples, we'll discuss different ways to detect the [Frida](#) instrumentation framework, which is used extensively in this guide. Other tools, such as [ElleKit](#) and [Xposed](#), can be detected similarly. Note that DBI/injection/hooking tools can often be detected implicitly, through runtime integrity checks, which are discussed below.

For instance, in its default configuration on a rooted device, [Frida](#) runs on the device as [frida-server](#). When you explicitly attach to a target app (e.g. via [frida-trace](#) or the [Frida REPL](#)), [Frida](#) injects a [frida-agent](#) into the memory of the app. Therefore, you may expect to find it there after attaching to the app (and not before). If you check `/proc/<pid>/maps` you'll find the [frida-agent](#) as [frida-agent-64.so](#):

```

bullhead:/ # cat /proc/18370/maps | grep -i frida
71b6bd6000-71b7d62000 r-xp /data/local/tmp/re.frida.server/frida-agent-64.so
71b7d7f000-71b7e06000 r--p /data/local/tmp/re.frida.server/frida-agent-64.so
71b7e06000-71b7e28000 rw-p /data/local/tmp/re.frida.server/frida-agent-64.so

```

The other method (which also works for non-rooted devices) consists of embedding a [frida-gadget](#) into the APK and forcing the

ALBERTO DIEGUEZ ALVAREZ

Editar perfil

Información Personal

Dirección de correo:

a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

Análisis Forense Informático (25-26)

Bastionado de Redes y Sistemas (25-26)

Hacking Ético (25-26)

Incidentes de Ciberseguridad (25-26)

Normativa de Ciberseguridad (25-26)

Puesta en Producción Segura (25-26)

|                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Explica qué es y para que se utiliza la herramienta Frida | Frida es un kit de herramientas de instrumentación de código dinámico y de código abierto, que funciona inyectando el motor de JavaScript QuickJS en el proceso instrumentado. Permite ejecutar fragmentos de JavaScript en aplicaciones nativas en Android e iOS. Se utiliza en análisis de seguridad por ejemplo para ver parámetros y valores en tiempo real...                                                    |
| Explica cómo se puede detectar en IOS (Frida Detection)   | Frida se ejecuta en el dispositivo con el proceso Frida-server que suele ser en un entorno jailbreak. Cuando por ejemplo se conecta a una aplicación la herramienta inyecta un agente llamado Frida-agent insertado dentro de un proceso. También se puede detectar revisando los puertos abiertos, Frida se suele comunicar mediante el puerto 27042, así que si ese puerto está activo, suele indicar su ejecución. |

## Apartado 2: CASB y MDM

Los servicios CASB (Cloud Access Security Broker) y las plataformas MDM (MobileDevice Management) desempeñan roles esenciales en la seguridad de la información en entornos empresariales modernos. La combinación de CASB y MDM proporciona un enfoque integral para abordar las amenazas modernas, asegurando tanto los datos en la nube como los dispositivos móviles, lo que es crucial para salvaguardar la integridad y confidencialidad de la información empresarial.

### 1. ¿Qué diferencias hay entre CASB y MDM (Mobile Device Management)?

Podemos diferenciar varias:

| CASB                                                                  | MDM                                                                                 |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Funciona como intermediario entre usuarios y los servicios en la nube | Gestiona y supervisa dispositivos móviles                                           |
| Protege datos en la nube con control de acceso, cifrado...            | Aplica políticas de seguridad sobre el dispositivo como apps permitidas, bloqueo... |
| Se enfoca en la seguridad de las apps cloud                           | Se enfoca en la seguridad del dispositivo                                           |
| Detecta accesos no autorizados                                        | Controla que el dispositivo cumpla con las normas, versiones...                     |

En resumen, el CASB se encarga de proteger la información en la nube mientras que el MDM se encarga de proteger los propios dispositivos que accedan a esa información.

### 2. ¿Con qué problemas se han encontrado las soluciones de MDM?

Las soluciones MDM se han encontrado con los siguientes problemas:

- Tienen un control limitado sobre las aplicaciones en la nube, es decir, no gestionan directamente las aplicaciones SaaS.
- Tienen dificultad con los dispositivos BYOD. Funcionan mejor con equipos corporativos.
- Problemas de privacidad, los empleados pueden percibir estas herramientas como intrusivas porque recopilan la información del dispositivo.
- Tienen una integración limitada con la nube ya que no siempre se conectan bien con otras opciones en la nube, y pueden dejarnos sin protección.

## Apartado 3: Diseño y capacidades de CASB

### 1. ¿Qué ventajas y desventajas en un entorno de CASB aporta tener un agente instalado en los terminales?

Las **ventajas** de tener un agente en un entorno CASB son:

- Mayor visibilidad del uso de aplicaciones en la nube desde el dispositivo.
- Mayor cumplimiento de normas de seguridad: Ayuda a las empresas a cumplir con la normativa de seguridad sobre el control de datos en la nube ya que facilita el seguimiento de la actividad en la nube.
- Más protección contra malware: Este puede bloquear malware además de otras amenazas en tiempo real, incluso antes de que el terminal sea infectado.

Las **desventajas** de tener un agente en un entorno CASB son:

- Requieren una instalación y mantenimiento para cada dispositivo.
- La experiencia de usuario puede verse afectada, es decir tiene un impacto en el rendimiento del terminal.
- El agente puede no ser compatible con todos los dispositivos.
- Al recopilar datos puede ser detectada como intrusiva por los usuarios.

### 2. ¿Qué problema supone cuando un usuario se va de la empresa en un entorno de BYOD (Bring Your Own Device)? ¿Qué mecanismos podríamos tener en nuestro entorno de CASB ideal para cuando un usuario abandone la compañía y siga siendo compatible con BYOD?

El problema que supone cuando un usuario se va de la empresa en un entorno BYOD es que el usuario se lleva el dispositivo personal y podría seguir accediendo a los recursos de la empresa si no se tiene control de permisos.

Como mecanismos en un entorno CASB ideal para BYOD cuando este abandone la compañía:

- Revocar rápidamente las credenciales, tokens de acceso...
- Borrar datos corporativos, pero sin borrar los personales.
- Tener una buena política basada en la persona no en el terminal, para tener una buena compatibilidad con BYOD.

### **3. ¿Qué capacidades adicionales podríamos añadir a nuestra solución de CASB?**

- Podríamos tener un control de apps en la nube no autorizadas.
- Tener un análisis del comportamiento de los usuarios.
- Supervisar y auditoría del cumplimiento normativo como GDPR...
- Bloquear o limitar el acceso a apps para reducir los riesgos de fuga de datos.
- Aplicar políticas de cifrado cuando se detectan datos críticos.
- Generar alertas automáticas ante la posibilidad de incidentes.